SAÉ 3.1 FA - Papillon

2025-2026

Michel TRAN, Rayan BISSON &
Emmanuel SRIVASTAVA-TIAMZON

Sommaire

I. Introduction
II. Objectif du projet
III. Choix ergonomique
IV. Wireframe
V. La structure du programme
VI. La base de données
VII. L'ajout de rappel
VIII. La modification
IX. La suppression
X. Le changement de position
XI. Gestions des messages d'erreurs
XII. Conclusions

I. Introduction

Situation d'apprentissage et d'évaluation pour le cours de développement au 3ème semestre du BUT Informatique

Dans le cadre de notre deuxième année de BUT informatique, pour le troisième semestre, nous avons eu l'opportunité de travailler sur un projet de développement en java nommé Papillon. Notre groupe est composé de Rayan BISSON, Michel TRAN et Emmanuel SRIVASTAVA-TIAMZON. Ce projet se concentre sur l'aspect Interface homme machine que l'on voit en cours (DEV-Interface Homme Machine) du développement d'une application basique mais utile, avec une attention particulière portée sur l'expérience utilisateur et l'ergonomie.

Papillon est un logiciel de gestion de rappel qui reste en permanence au premier plan comme un papillon collé sur le moniteur. Il affiche une liste de rappels que l'utilisateur souhaite avoir sous les yeux, qu'il peut personnaliser par thèmes de couleurs, par son ordre, leurs titres et leurs contenus.
L'objectif est de fournir à l'utilisateur un accès visuel constant et immédiat à ses notes et tâches importantes, sans pour autant entraver l'utilisation des autres fenêtres. Pour cela, une contrainte majeure du développement a été de minimiser l'espace occupé par l'application tout en assurant une ergonomie optimale.
L'application permet aux utilisateurs de gérer une liste de rappels, chacun défini par un titre (limité à 50 caractères), un contenu (4 lignes maximum de 50 caractères chacune) et un thème visuel distinctif. Nous avons mis en place au moins cinq thèmes de couleurs différentes.

Ce projet a été développé en Java en suivant les consignes du professeur de développement Luc Hernandez, notamment en n'utilisant que les classes originales et celles vues en cours.

Une exigence fondamentale de ce projet réside dans la gestion et la persistance des données. Les rappels et leur ordre personnalisé doivent être conservés d'une session à l'autre, nécessitant l'intégration d'un serveur de base de données.

II. Objectif du projet

Développement d'une solution de rappels persistantes et ergonomiques.

1. L'objectif fonctionnel

En premier temps, en tant que développeurs, nous avons un objectif fonctionnel, qui est de développer une application avec certains critères. Ici ces critères sont :

- Digitaliser le Post-It : L'ambition principale est de créer un outil de gestion de rappels qui imite le comportement d'un "papillon" (post-it) collé sur un écran.
  - Permanence : L'application reste toujours au premier plan pour ne jamais oublier une tâche.
  - Discrétion : Occuper un espace minimal pour ne pas gêner le travail sur les autres fenêtres.
  - Gestion complète : Permettre de créer, modifier, supprimer et organiser des rappels simplement (CRUD).

2. L'objectif ergonomique (UI/UX)

En second temps, il faut comprendre qu'un développeur ne doit pas seulement au côté pratique et fonctionnel mais à une ergonomie opérationnelle. Chose qui est beaucoup négligée au sein des développeurs :

- Une Interface Intuitive (IHM) : L'objectif est de réduire la charge mentale et la frustration de l'utilisateur grâce à une interface soignée.
  - Hiérarchie Visuelle : Mise en avant des titres et des actions principales (boutons en haut) pour une lecture rapide.
  - Feedback Immédiat : Réactivité des boutons au survol pour confirmer les actions de l'utilisateur.
  - Design "Flat" & Pastel : Choix de tons jaunes/blancs évoquant le post-it papier, assurant une lisibilité optimale et un confort visuel, même sur une longue durée.

3. L'objectif technique et pédagogique

Enfin, un développeur est un développeur organisé, qui sait professionnellement présenter le rendu :

- Défi Technique & Collaboration Mettre en pratique les acquis du BUT Informatique dans un contexte quasi-professionnel.
  - Langage Java pur : Utilisation exclusive des classes standard et originales.
  - Persistance des données : Connexion à un serveur de base de données pour sauvegarder les rappels et leur ordre entre les sessions.
  - Travail d'équipe : Gestion du versionning via Gitea et respect des normes de code (Javadoc, organisation en packages).

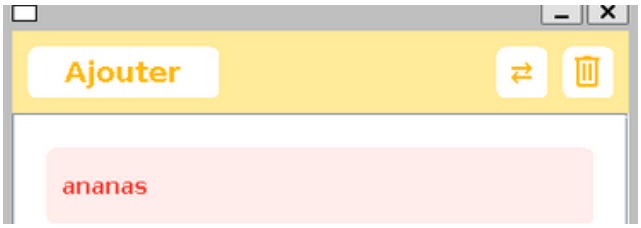
III. Choix ergonomique

La réussite d'une bonne application repose sur un choix ergonomique irréprochable. L'objectif est de réduire la frustration de l'utilisateur et ses erreurs pour que l'utilisation de l'outil devienne un réflexe et que l'application soit intuitive.
Il est donc important de concevoir une bonne interface homme-machine à travers une bonne UX (User Experience) ainsi qu'une bonne UI (User Interface). Pour cela nous avons étudié en amont le sujet et nous avons déduis plusieurs points clés sur lesquelles se concentrer afin d'obtenir une bonne UX et UI.
Voici les choix ergonomique ainsi que nos explications :

La User Interface :

Une hiérarchie claire

Afin d'obtenir une bonne hiérarchie visuelle les titres doivent être gros et les actions principales doivent etre mise en avant. C'est pour cela que nous avons mis les titres des rappels en gras. Nous avons aussi mis les actions principales en avant en les mettant en haut de la fenetre avec une taille suffisante pour attirer l'attention de l'utilisateur.



Un feedback immédiat

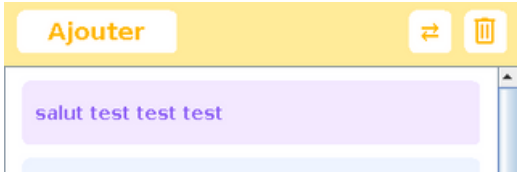
Le feedback immédiat est essentiel pour concevoir une bonne interface utilisateur. Il permet de maintenir l'utilisateur informé de ce qui se passe dans l'application à chaque interaction. C'est pourquoi nous avons ajouté sur chaque bouton différents effets visuels en fonction des actions de la souris : par exemple, la couleur change au survol et se modifie à nouveau lors du clic.

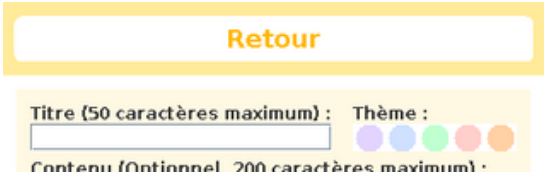
Bouton pas survolé

Bouton survolé

Cohérence

Dans le but d'obtenir une bonne cohérence entre les différentes pages, nous avons décidé de placer les boutons importants en haut de la fenêtre. Cela permet à l'utilisateur de les repérer rapidement et de naviguer plus facilement, sans avoir à chercher où se trouvent les actions essentielles.





L'esthétique et le design

Qui dit rappel dit post-it, c'est pour cela que nous voulions que notre application évoque l'apparence d'un post-it, ainsi nous avons choisi des tons jaunes et blancs pour l'ensemble des fenêtres. La majorité des couleurs sélectionnées sont des teintes pastel, elles permettent d'obtenir une interface lisible et agréable. Comme l'application est destinée à rester affichée en permanence au premier plan, il était essentiel qu'elle reste confortable à regarder lors d'une utilisation prolongée.
Concernant le design, nous avons opté pour un flat design, afin de garantir une meilleure lisibilité et de faciliter la navigation grâce à une interface simple.

La User Experience:

La loi de Hick

Comme l'explique ce que nous avons vu en cours : "Le temps pour prendre une décision augmente avec le nombre et la complexité des choix."
— William Edmund Hick.

Nous avons donc fait en sorte de réduire au maximum les cliques de souris et de choix pour effectuer une fonctionnalité

La loi de Fitts

Comme l'explique ce que nous avons vu en cours : "Le temps pour atteindre une cible est une fonction de la distance à la cible et de la taille de la cible."
— Paul Fitts.

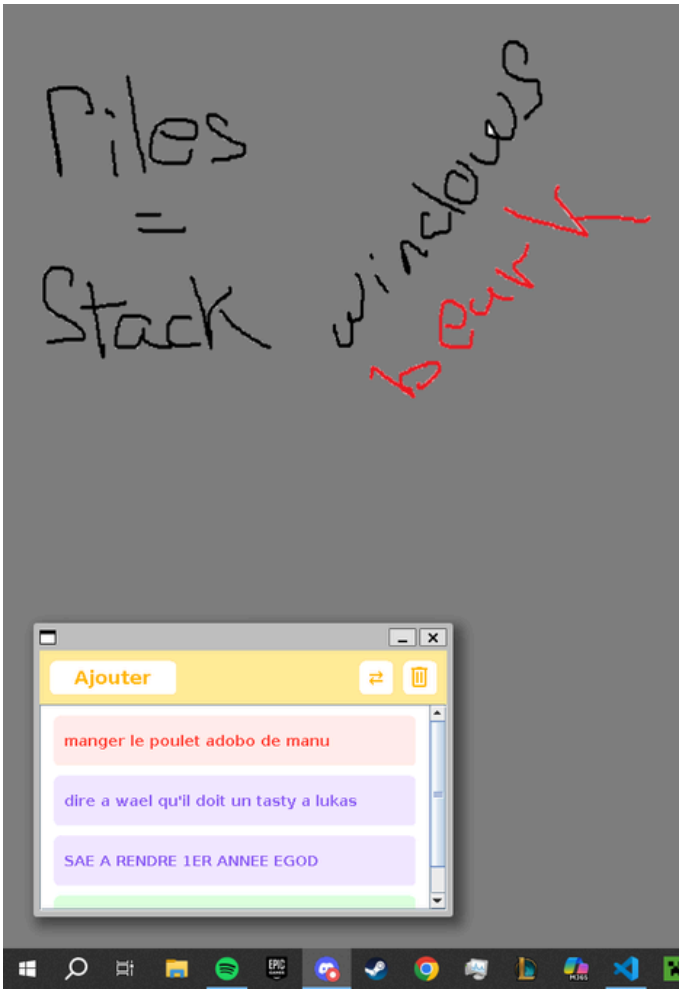
Nous savons aussi que les quatre coins d'une fenêtre sont les zones les plus rapides à atteindre avec la souris. Ainsi nous avons donc décider de placer les boutons important dans les coins de la fenêtre. De plus, nous avons agrandi la taille de certains boutons pour réduire le temps pour atteindre une cible.

La loi de Jacob

Comme l'explique ce que nous avons vu en cours : "Les utilisateurs passent la plupart de leur temps sur d'autres sites/applications. Cela signifie qu'ils préfèrent que votre site fonctionne de la même manière que tous les autres sites qu'ils connaissent déjà." — Jakob Nielsen.

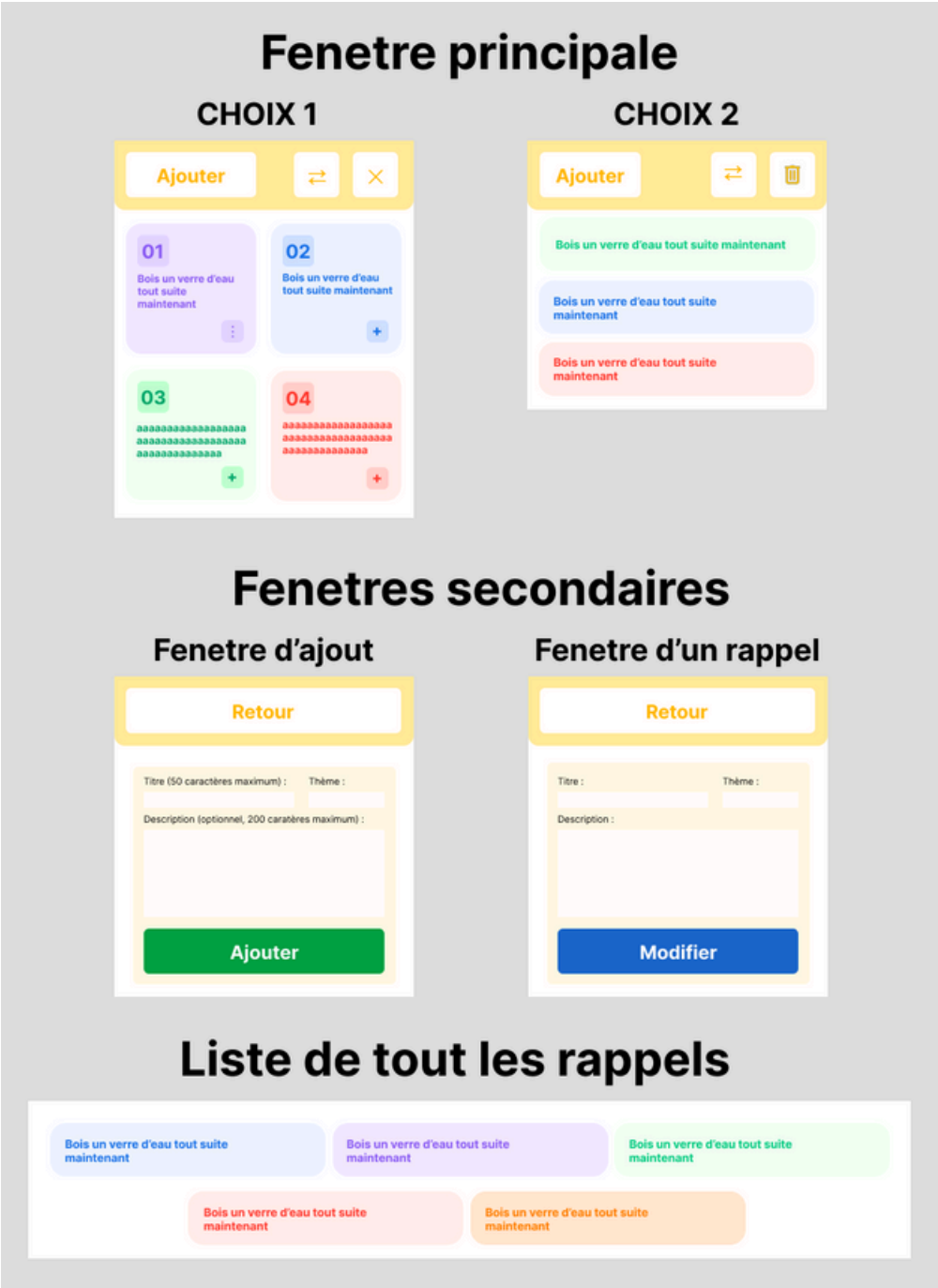
À l'origine, nous voulions ajouter un bouton pour quitter l'application. Cependant, la majorité des applications utilisent le bouton de fermeture de fenêtre par défaut. Les seuls cas où un bouton « quitter » est présent c'est lorsque que les applications sont en mode plein écran, ce qui n'est pas notre cas. Nous avons donc décidé de ne pas en ajouter, afin que l'application fonctionne de la même manière que toutes les autres applications.

<u>Le placement de la fenetre :</u>



Pour l'ergonomie de la fenêtre, sa position était un point clé. A l'origine nous voulions la positionner en haut à droite, mais nous nous sommes rendu compte que cela pouvait être dérangeant, car le bouton de fermeture des fenêtres est généralement situé à cet endroit. Le placement en haut posait également problème, puisque les onglets des navigateurs internet s'y trouvent souvent. Nous avons donc choisi la position en bas à gauche afin de gêner le moins possible l'utilisation des autres fenêtres.

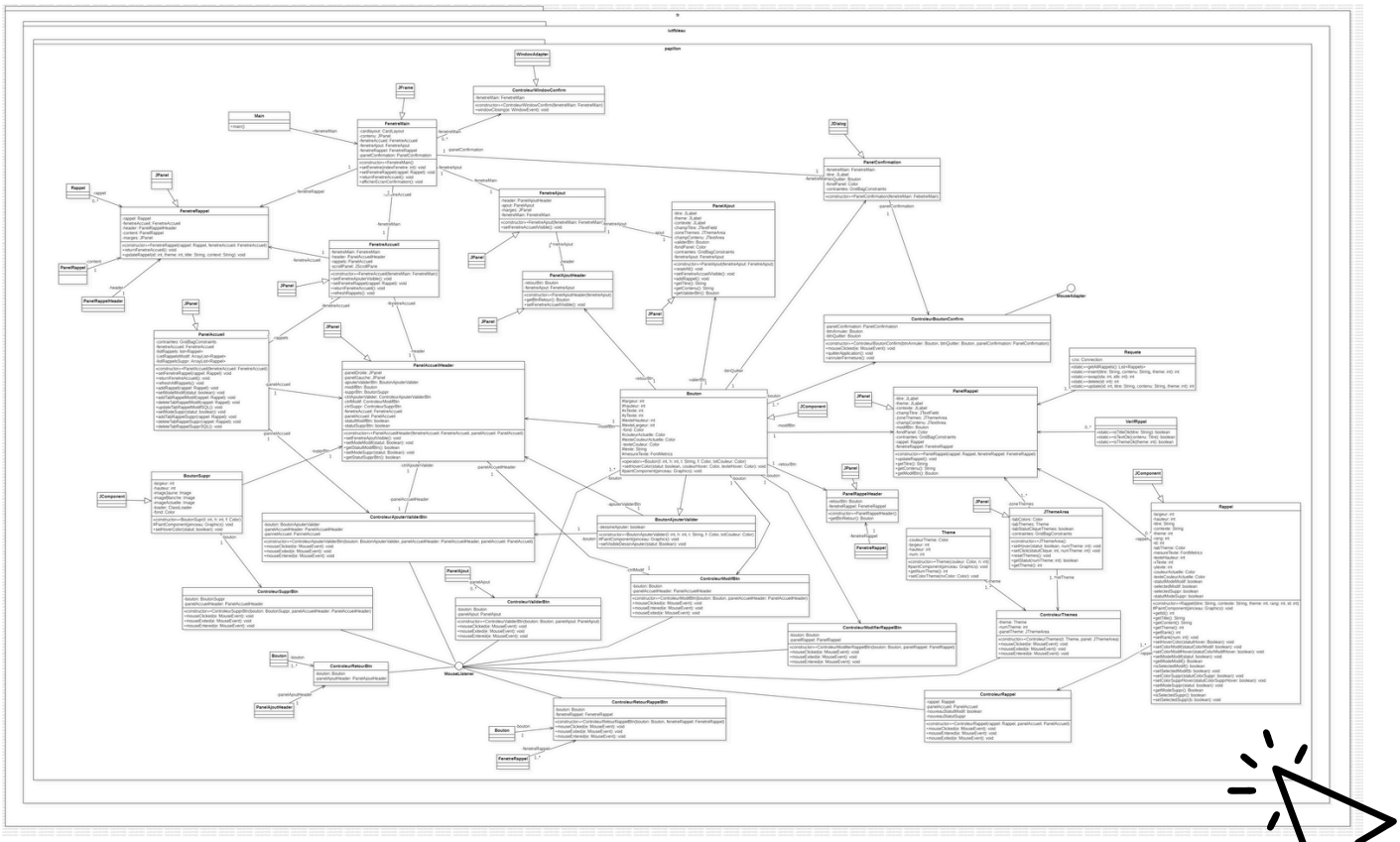
IV. Wireframe



Voici le wireframe que nous avons fait sur Figma avant de nous lancer dans le code. A l'origine nous voulions partir sur le choix 1 pour la fenêtre principale. Cependant pour des raisons d'ergonomie nous nous sommes dirigé vers le choix 2.

V. La structure du programme

Dans un projet de développement, que ça soit d'application ou web, un diagramme de classes est toujours utile pour le(s) développeur(s) pour la conception en tant que visualisation de différentes classes utilisées. Pour l'utilisateurs, il peut être une aide sur la compréhension du code et de la composition des classes, de leur méthodes et de leur attributs.
Notre diagramme de classe est très détaillé, donc révèle notre code entièrement en UML.

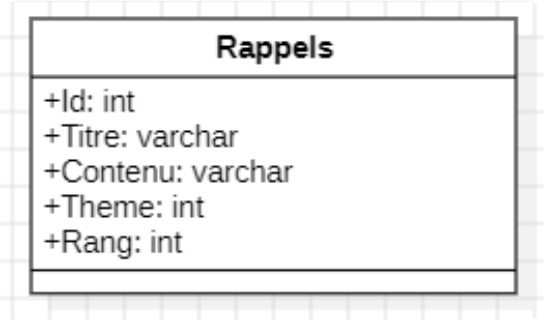


Puisque la lisibilité n'est pas optimale, nous vous donnons l'option de cliquer sur l'image pour pouvoir être redirigé vers l'image original, qui est de meilleur qualité. Si besoin il est consultable dans le dossier **res**.

VI. La base de données

Id	**int(11)**	La clé primaire, l'identifiant qui permet de cibler un rappel précis pour les opérations de modification ou de suppression (CRUD).
Titre	**varchar(50)**	Stocke le titre du rappel. Limitée à 50 caractères conformément aux contraintes. Ce champ est obligatoire (NOT NULL).
Contenu	**varchar(200)**	Stocke le texte du rappel. La taille de 200 caractères car : 4 lignes de 50 caractères maximum. Ce champ accepte la valeur NULL car un rappel peut être vide.
Theme	**int(30)**	Stocke l'identifiant visuel du thème choisi (ex: 0 = violet, 1 = bleu, 2 = vert, 3 = rouge, 4 = orange). Il est stocké en tant que integer.
Rang	**int(11)**	Stocke le rang actuel du Rappel, valeur de type int qui définit l'ordre d'affichage rappels à l'écran. L'utilisateur peut interchanger ses rappels comme bon lui semble.

Ci-dessous, est modélisé un diagramme de classe UML, représentant la base de données utilisée pour les Rappels affichés lors de l'exécution du programme. Elle est utilisée lorsque l'on visionne les rappels présents, lorsque l'on en modifie un, lorsque l'on en supprime un ou encore lorsque l'on en déplace un.

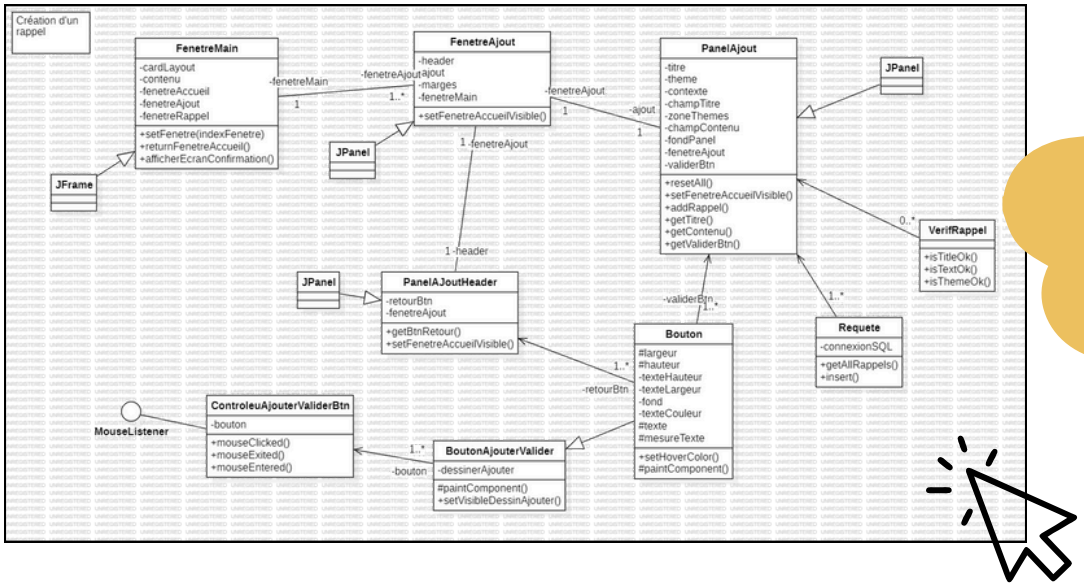


VII. L'ajout de rappel

L'interface graphique de la fenêtre d'ajout

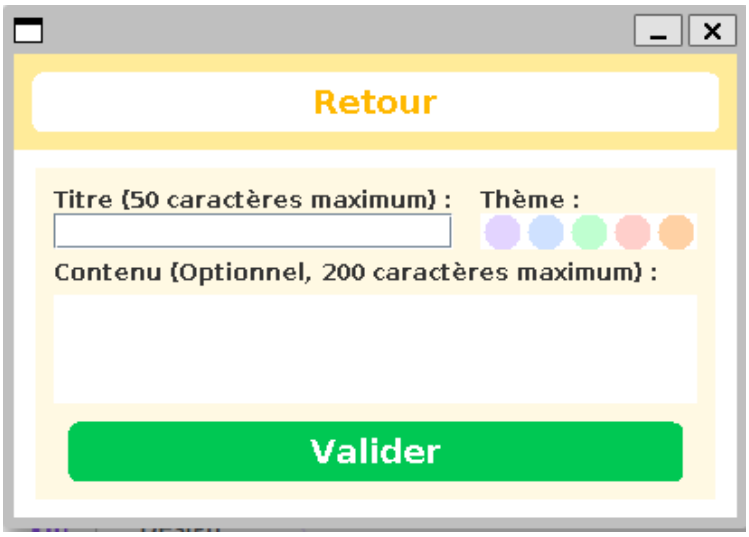
On s'attaque au **C de la méthode CRUD**. Composée de plusieurs classes, pour le header, les thèmes, et les différents choix de personnalisation possible (personnalisation limitée du titre et du contenu). Elle a été pensée sur **Figma** lors de notre phase de test.

Pour pouvoir ajouter des rappels, tout a été optimisé, le nombre de cliques, le temps pris entre chaque action. Pour respecter les contraintes imposées, on a écrit une classe composée de méthodes qui vérifie ces mêmes contraintes pour le titre et le contenu (**VerifRappel**). Par exemple si l'utilisateur insère plus de 50 caractères ou rien dans le titre puis le valide, alors, un pop up s'affichera et préviendra de l'erreur sans enregistrer le rappel. Pareil pour le contenu.



Dû au fait que la lisibilité soit mauvaise, vous pouvez cliquer sur le diagramme ou aller dans le dossier **res**.

Au-delà des erreurs de saisie, l'application doit aussi gérer les imprévus techniques lors des échanges avec la base de données. Pour sécuriser les requêtes **(insertion, mise à jour, suppression)**, nous avons encapsulé nos appels à la base de données dans des structures de contrôle (blocs try-catch).

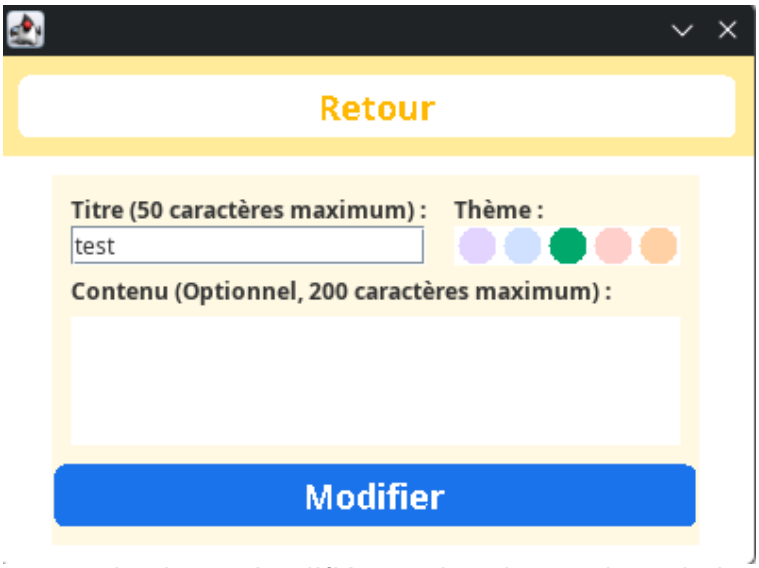


Cela permet d'intercepter les erreurs critiques, comme une perte de connexion au serveur ou un échec de requête SQL. Au lieu de faire planter l'application brutalement, le programme capture l'exception et affiche une alerte utilisateur sécurisée. Cela garantit une **expérience fluide** même en cas de dysfonctionnement technique.

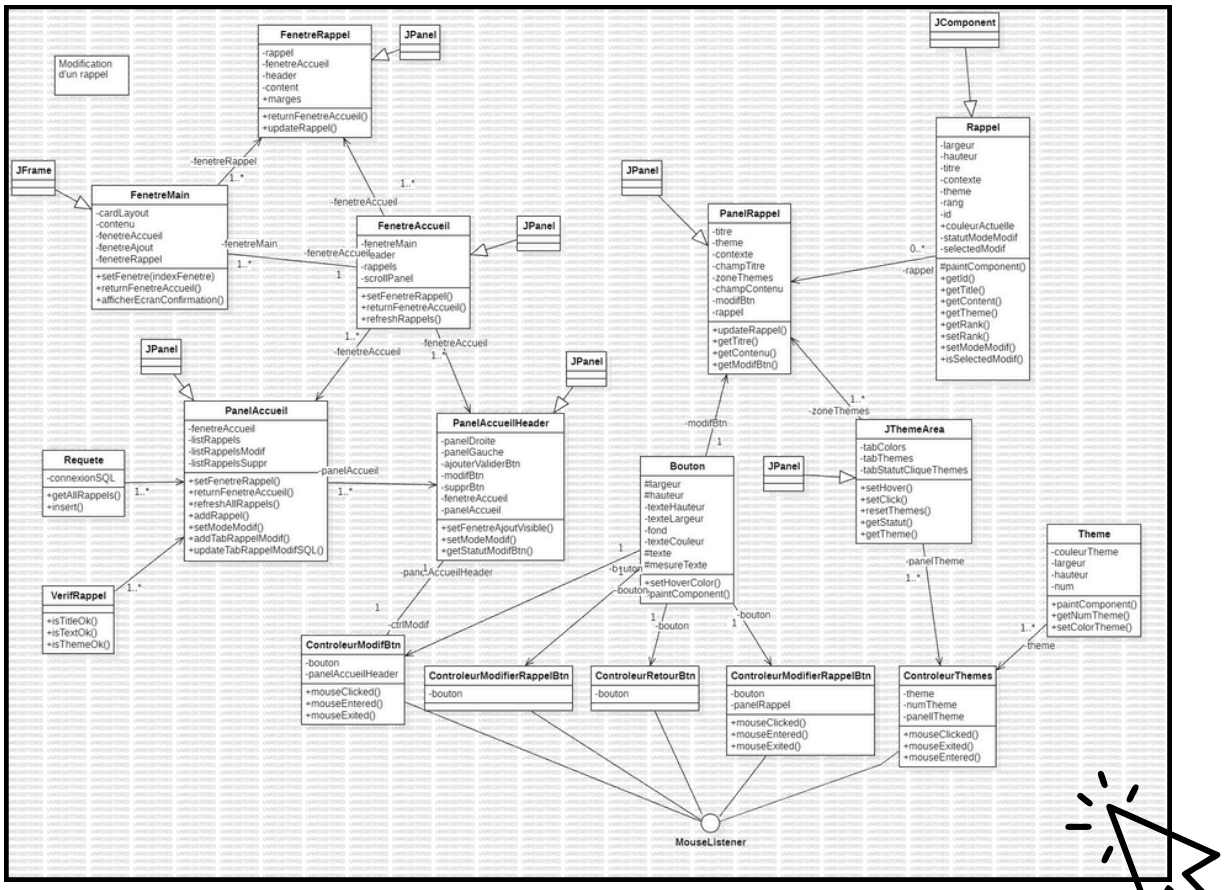
VIII. La modification

L'interface graphique de la fenêtre de modification

Pour pouvoir modifier des rappels, on a copié la même façon de faire que l'ajout de rappel pour ne pas déstabiliser l'utilisateur avec une nouvelle interface qui brise les conventions que l'on a mit en place. Il y a donc les mêmes contraintes que sur la fenêtre d'ajout. Pour y accéder il faut cliquer sur un rappel.



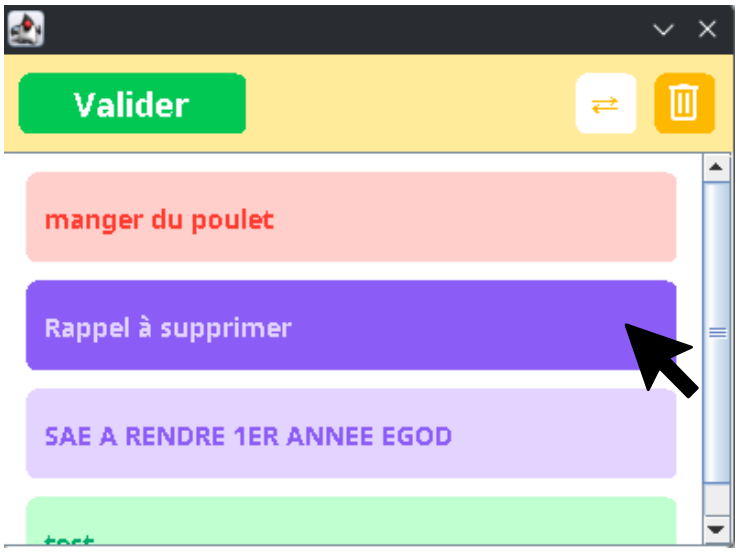
Ci-dessous, est le diagramme de classe simplifié avec les classes, les relations, les attributs pour la modification d'un rappel que vous retrouverez dans le dossier **res** et en cliquant dessus.



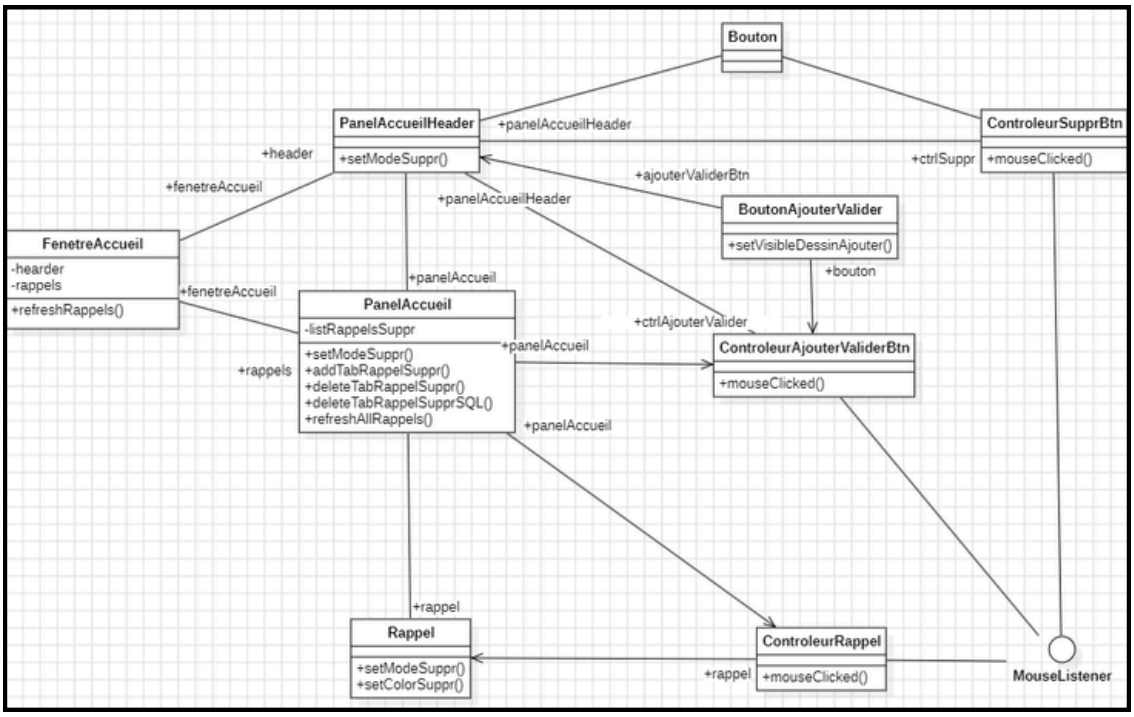
IX. La suppression

L'interface graphique de la fenêtre lors de la suppression de rappel

La suppression d'un rappel, prend en considération des classes de contrôleur, de type JPanel, de vue mais aussi de requête SQL. à partir de la fenêtre principale il faut cliquer sur l'icone *poubelle* et ensuite cliquer sur un ou plusieurs rappels puis *valider*. Nous avons fais en sorte à ce que ça soit le plus intuitif possible avec les couleurs et les icônes.



Ci-dessous, est le diagramme de classe simplifié avec les classes, les relations, les attributs pour la suppression d'un rappel que vous retrouverez dans le dossier **res** et en cliquant dessus.



Explication de la fonctionnalité de suppression :

Lorsqu'on souhaite effectuer une suppression d'un ou plusieurs rappels, on clique sur le bouton pour activer le mode "Suppression". Cela ce fait grâce a la classe "ControleurSupprBtn" qui va appeler la méthode "setModeSuppr" de la classe "PanelAccueilHeader", cette méthode va modifier l'affichage du bouton de la classe "BoutonAjouterValider" pour afficher un bouton "Valider", cette méthode va aussi appeler la méthode "setModeSuppr" de la classe "PanelAccueil" qui va appeler la méthode "setModeSuppr" de la classe "Rappel". Cela va permet a la classe "ControleurRappel" de pouvoir appeler les méthodes suivantes de la classe "PanelAccueil" :

- addTabRappelSuppr, méthode permettant d'ajouter plusieurs rappels à une liste qui sera utilisée pour une requête SQL plus tard.
- deleteTabRappelSuppr, méthode permettant de supprimer un élément de cette liste et de réorganiser cette liste.

Une fois les rappels sélectionnés, on clique sur le bouton de la classe "BoutonAjouterValider" pour que son controleur appelle les méthodes "deleteTabRappelSupprSQL" et "refreshAllRappels".

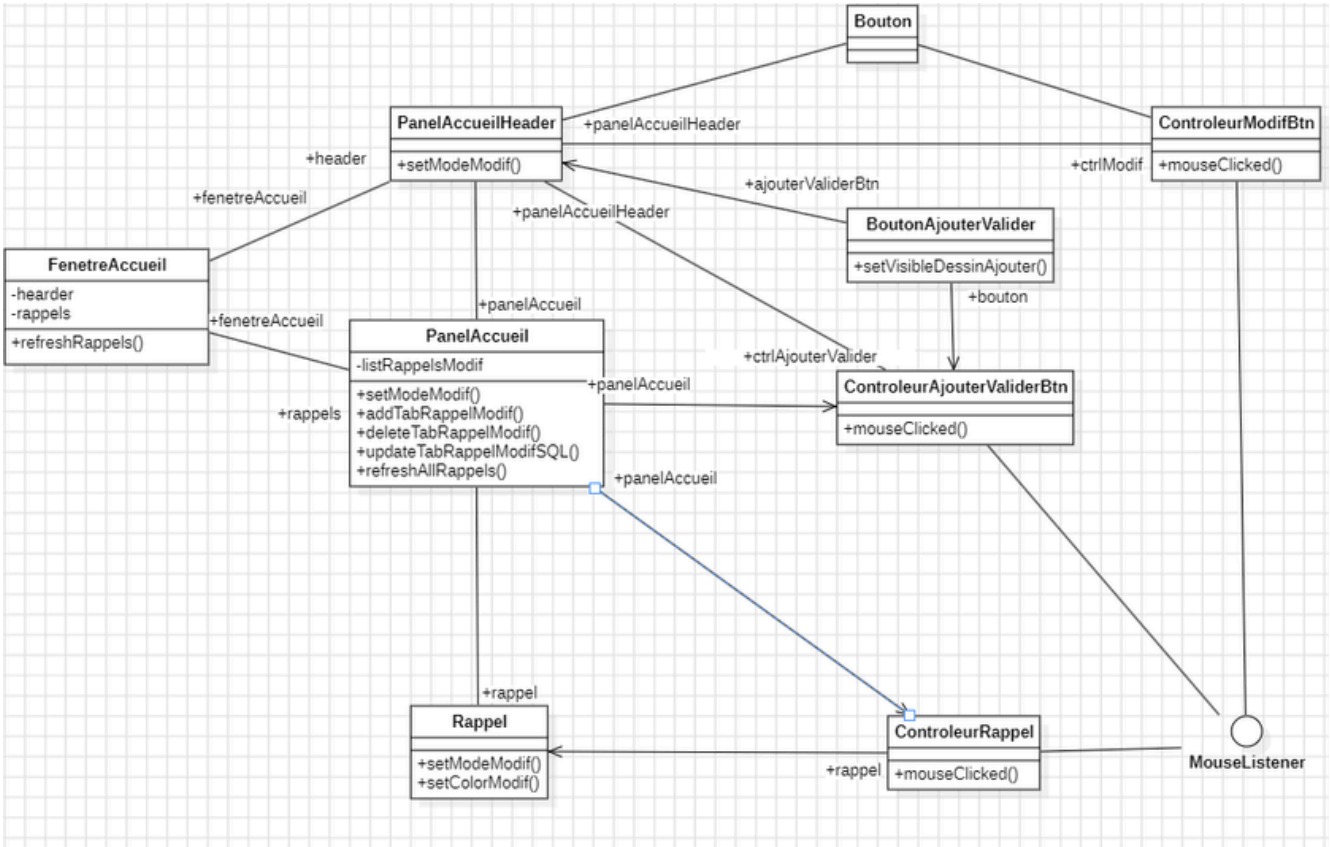
X. Le changement de position

Le tri des rappels

Composé de plusieurs classes, on utilise le header pour accéder à l'option de changement de position des rappels.

Pour pouvoir modifier des rappels, on a copié la façon de faire de l'ajout de rappel pour ne pas déstabiliser l'utilisateur avec une nouvelle interface qui brise les conventions que l'on a mit en place. Il y a donc les mêmes contraintes que sur la fenêtre d'ajout. En premier temps, il faut sélectionner un rappel puis un autre pour les interchanger de place. Leur place/rang est enregistré sur la base de donnée distante pour que l'utilisateur puisse retrouver son tri à chaque lancement de l'application.

Lorsque l'on survole un rappel, la couleur est un peu plus intensifiée pour montrer à l'utilisateur lequel il choisit. Le bouton valider est en vert car il était anciennement blanc, ce qui rendrait cette fonctionnalité non intuitive (nous l'avons remarqué en faisant tester notre application à notre entourage).



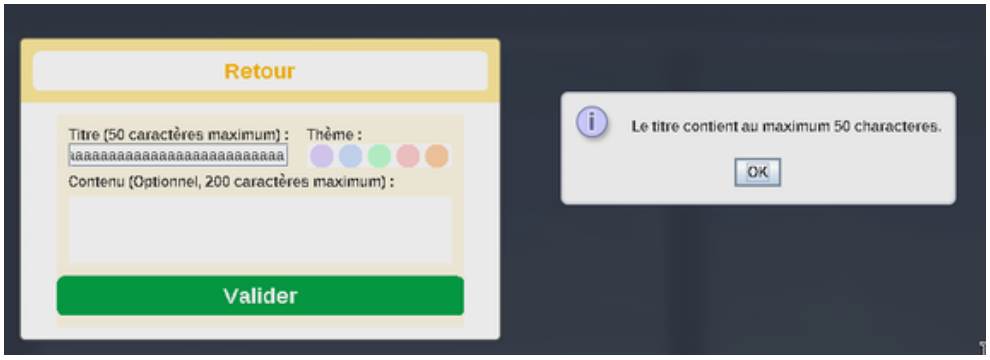
Explication de la fonctionnalité de tri :

Le fonctionnement du tri est presque identique à celui de la suppression, cependant la liste qui stocke les rappels à échanger est limitée à deux rappels. Lorsqu'on rajoute une troisième rappel la méthode "addTabRappelModif" va supprimer le premier rappel de la liste, puis deplacer le deuxieme à la premiere position et enfin ajouter le troisième rappel à la deuxieme postion.

XI. Les messages d'erreur

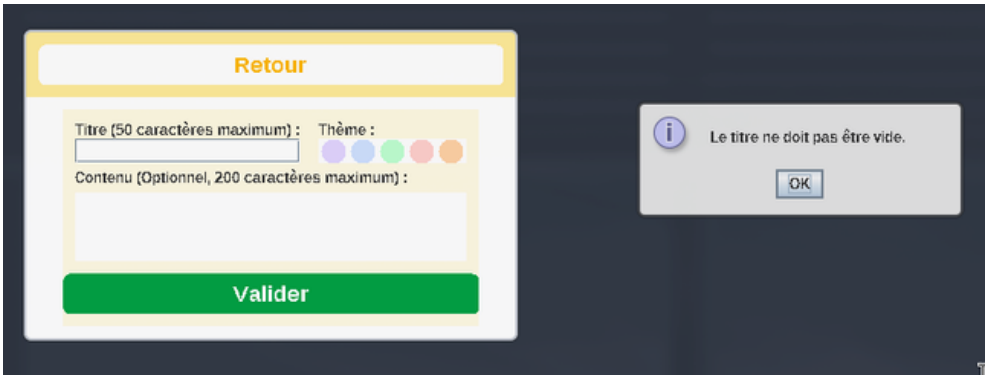
Nous affichons à l'écran de l'utilisateur une variété de messages d'erreur qui concernent l'insertion (et donc par extension la modification) des rappels conformément à l'énoncé de notre projet.

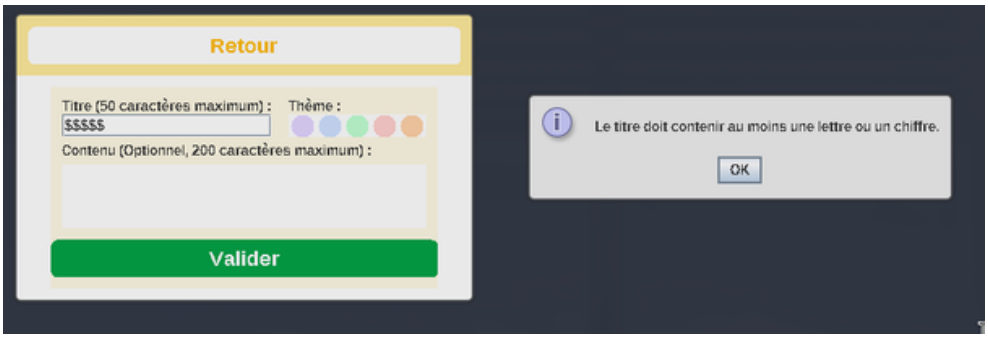
Le titre



L'utilisateur ne peut pas créer un rappel avec un titre de plus de 50 caractères, sinon il obtient le message d'erreur ci-contre.

Ce message d'erreur est assez explicite, l'utilisateur n'a pas le droit de créer de rappel sans titre valide.

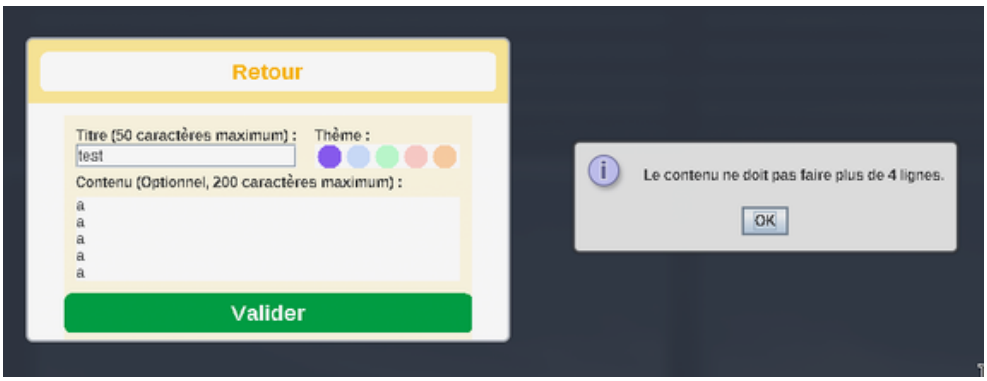




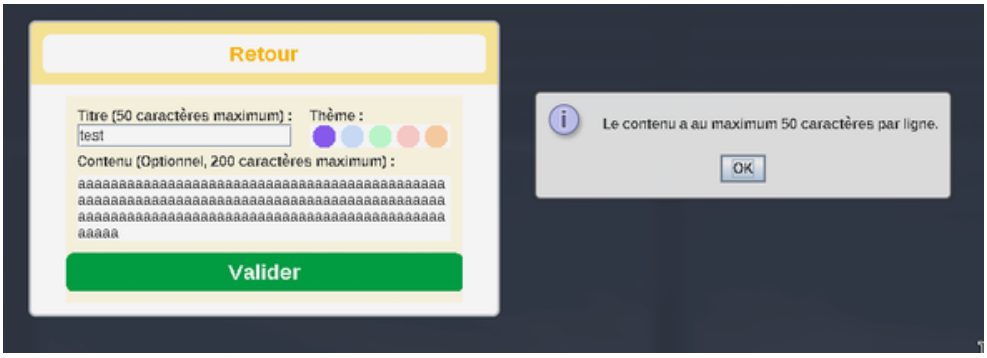
Ce message s'affiche si l'utilisateur ne met que des caractères spéciaux sans mettre de lettre ou de chiffre. On note tout de même que à partir du moment ou il y a un caractère alphanumérique dans le titre, l'utilisateur peut mettre n'importe quoi en tant que titre.

Le contenu

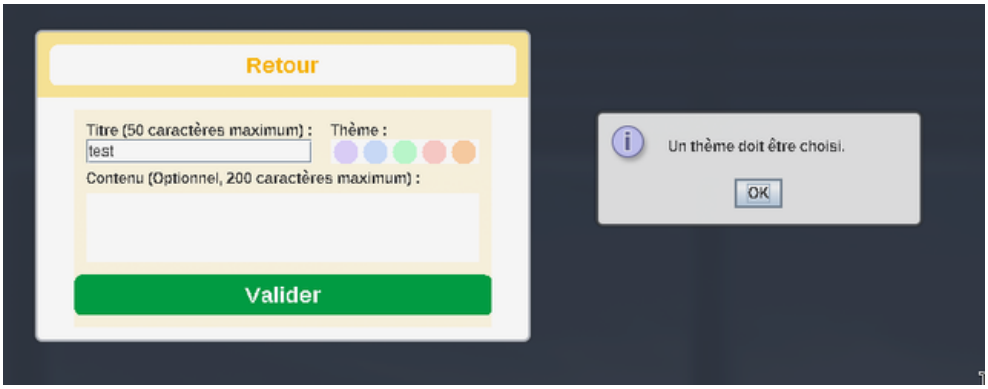
Ce message d'erreur est assez explicite, on vérifie que l'utilisateur sépare bien le contenu en 4 lignes de 50 caractères différents.



Ce message s'affiche si l'utilisateur écrit plus de 50 caractères sans passer à la ligne suivante.



Le thème



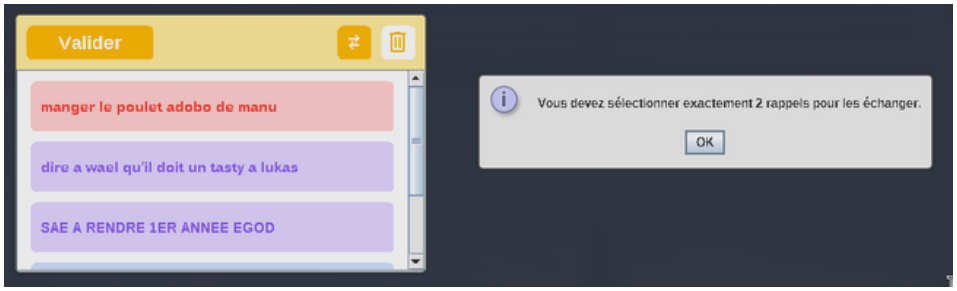
Il est important de préciser que la vérification que l'utilisateur ait bien choisi un thème ne s'effectue pas dans le cas de la modification d'un rappel, car si le rappel est inséré l'utilisateur a forcément déjà choisi un thème.

Le mode échange de rappels

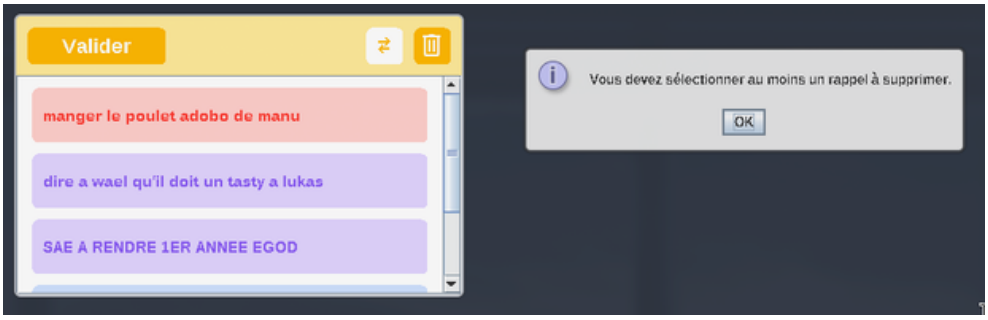
Ce message d'erreur s'affiche dans les cas suivants :

- Aucun rappel n'est sélectionné
- 1 seul rappel est sélectionné

C'est le seul cas qui peut causer une erreur dans le programme donc le seul qui a un message d'erreur pour l'échange de rappels.



Le mode suppression de rappels



Ce message d'erreur s'affiche quand aucun rappel n'est sélectionné pendant le mode suppression.

XII. Conclusion

Conclusions personnelles :

- **Michel :**

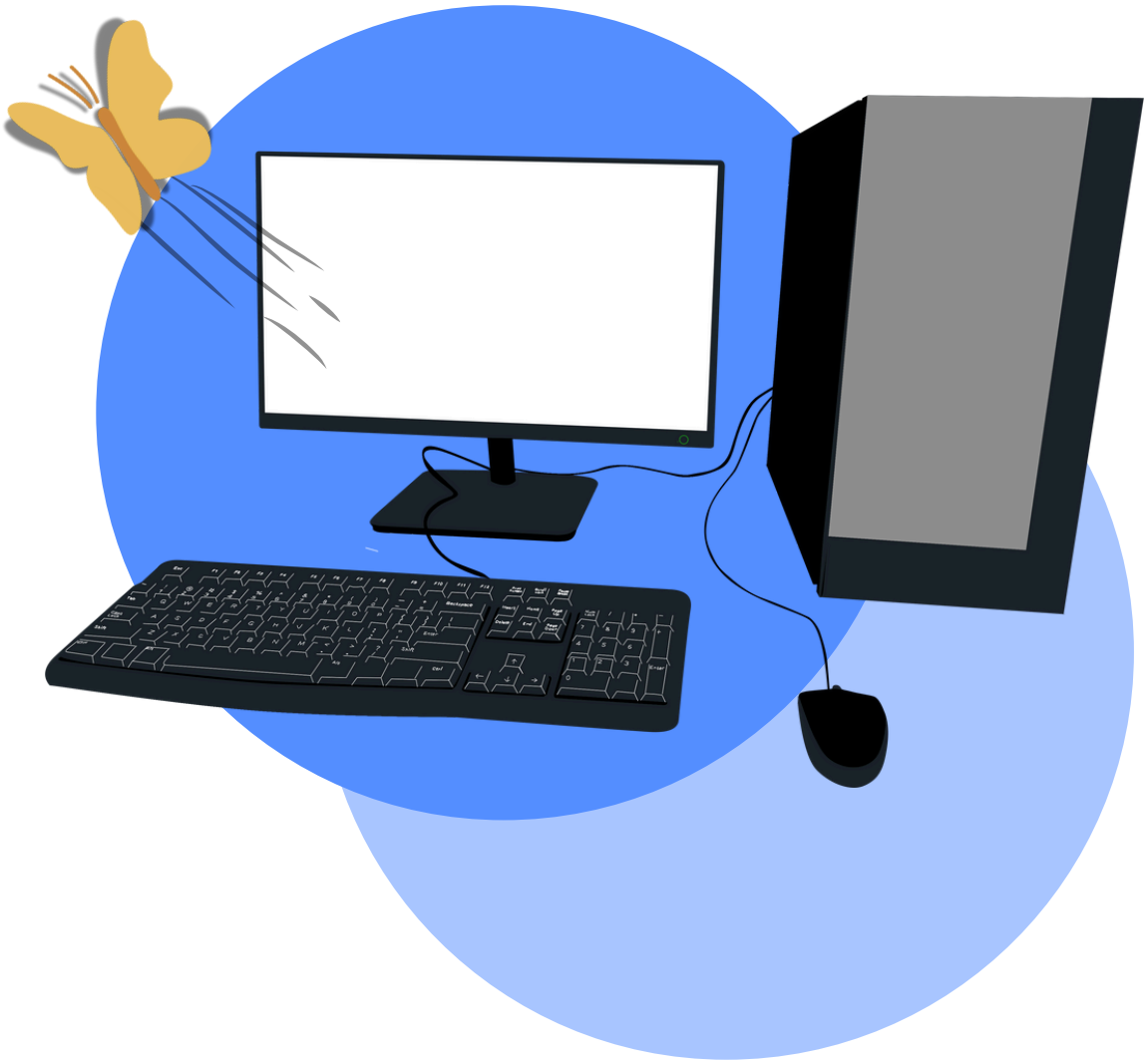
Durant ce projet, je me suis beaucoup amusé et je l'ai vraiment apprécié. Au début, j'ai rencontré quelques difficultés à me remettre au Java étant donné que le dernier projet de Java datait de la première année, mais au final je m'y suis vite réhabitué.
J'ai passé pas mal de temps sur l'identité visuelle de l'application (choix des couleurs et forme des rappels), même si ça ne sera pas forcement noté, je voulais quand même y apporter ma touche créativité à ce projet.
Concernant les choix ergonomiques j'ai vraiment essayé de me mettre le plus possible à la place de l'utilisateur pour que l'application soit intuitive, j'ai aussi reçu le feedback de certains élèves concernant certains choix ergonomiques qui m'ont permis de les régler.
Donc voila, dans l'ensemble j'ai beaucoup aimé passer du temps sur la partie front end du projet.

- **Rayan :**

Durant ce projet ma partie a plus concerné le back-end, donc la gestion des données de l'application et les finitions et tests à la fin du projet. Pour être honnête, je regrette légèrement de ne pas avoir pu faire plus d'efforts pendant le développement de l'application, car le travail de manipulation des fenêtres et de front-end était à mon sens infiniment plus élevé que le travail de back-end. Michel est arrivé avec des idées d'apparence de l'application incroyables et une mise en oeuvre de ces idées avec une rapidité exceptionnelle, et face à cela je me sentais obligé de suivre le pas. Le Makefile était également très compliqué à faire, avec notamment la compilation de l'archive jar qui devait inclure le package de la base de données et les images nécessaires à l'application, mais j'ai quand même réussi à tout faire fonctionner. Les efforts fournis par Emmanuel pour ses petites touches dans le programme et également la confection du rapport et du diagramme UML complet ne sont aussi pas à sous-estimer. Le travail de front-end ne m'intéresse vraiment pas et c'est pour ça que j'applaudis d'autant plus mes camarades qui ont tout mis en oeuvre pour que ce projet soit aussi beau qu'il l'est aujourd'hui. La raison de pourquoi je reste dans le BUT Informatique de Fontainebleau est que à chaque fois que nous faisons un nouveau projet, je me dis que c'est le meilleur que j'aurais pu faire sur le moment, et donc qu'à chaque fois je fais un projet meilleur que le précédent. Ce projet a encore une fois été une énorme réussite d'après-moi, et le résultat de notre travail d'équipe a primé et c'est presque émouvant de voir une application aussi réussie.

- **Emmanuel :**

Ce projet de développement d'une application de gestion de rappels en Java a été une expérience formatrice. J'ai pu y appliquer mes connaissances en contribuant activement au développement des fonctionnalités clés, des contrôleurs et des modèles, tout en veillant à la fiabilité du code.
En parallèle, la réalisation des diagrammes de classes et la rédaction de ce rapport m'ont permis de consolider ma vision globale du développement logiciel. Au final, je suis satisfait d'avoir pu aboutir à une solution fonctionnelle et ergonomique, acquérant ainsi des bases solides pour mes futurs projets.



Ce projet a été réalisé dans le cadre d'une situation d'apprentissage et d'évaluation au sein de la formation de BUT informatique à Fontainebleau dans un but d'apprentissage uniquement. Les auteurs sont :

Michel TRAN, Rayan BISSON & Emmanuel SRIVASTAVA-TIAMZON